

Группа	Z3342	К работе допущен	_____
Студент	Погорелко В. А.	Работа выполнена	_____
Преподаватель	Маслов И. Д.	Отчёт принят	_____

Рабочий протокол и отчёт по лабораторной работе №4

[C++ & UNIX]: C++ PROCESSES / THREADS

1. Цель работы

Познакомить студента с принципами параллельных вычислений. Составить несколько программ в простейшими вычислительными действиями, чтобы освоить принципы параллельных вычислений (когда одни алгоритмы зависят / не зависят от других).

2. Задачи

1. (C++ SEQUENCE) Последовательные вычисления. Требуется последовательно выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1. Выполнить последовательно на 10 000 итераций и 100 000 итераций

Формула 1: $f(x) = x^2 - x^2 + x \cdot 4 - x \cdot 5 + x + x$

Формула 2: $f(x) = x + x$

Вывести длительность выполнения всех 10 000 итераций и 100 000 итераций в сек.

2. (C++ THREADS) Параллельные вычисления через потоки. Требуется параллельно (насколько возможно с помощью потоков) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1
3. (C++ PROCESS) Параллельные вычисления через процессы. Требуется параллельно (насколько возможно с помощью процессов) выполнить вычисления по формуле 1, вычисления по формуле 2, после чего выполнить вычисления по формуле 3, которые выглядят следующим образом: результат вычислений 1 + результат вычислений 2 – результат вычислений 1
4. (LOG) Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc) Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).

3. Решение

1. (C++ SEQUENCE) Последовательные вычисления

В папке lab_04/src создаем файл sequence_test.cpp:

```
sequence_test.cpp

#include <iostream>
#include <chrono>
using namespace std;
float formula1(float x) { return x*x-x*x+x*4-x*5+x+x; }
float formula2(float x) { return x+x; }
int main() {
    int iterations[] = {10000, 100000};
    float x = 0.13;
    for (int n : iterations) {
        float accum = 0.0;
        auto start = chrono::high_resolution_clock::now();
        for (int i = 0; i < n; i++) {
            float res1 = formula1(x);
            float res2 = formula2(x);
            float res3 = res1 + res2 - res1;
            accum += res3;
        }
        auto end = chrono::high_resolution_clock::now();
        chrono::duration<double> delta = end - start;
        cout << "Iterations: " << n << endl;
        cout << "Time: " << delta.count() << " seconds" << endl;
        cerr << "Accumulated value: " << accum << endl;
    }
    return 0;
}
```

После компиляции и исполнения программы получаем следующие результаты:

```
Iterations: 10000
Time: 0.00011551 seconds
Accumulated value: 2600.09
Iterations: 100000
Time: 0.000660054 seconds
Accumulated value: 25980.4
```

2. (C++ THREADS) Параллельные вычисления через потоки.

В папке lab_04/src создаем файл threads_test.cpp:

```

threads_test.cpp
#include <iostream>
#include <chrono>
#include <thread>
using namespace std;
float formula1(float x) { return x*x-x*x+x*4-x*5+x+x; }
float formula2(float x) { return x+x; }
int main() {
    int iterations[] = {10000, 100000};
    float x = 0.13;
    for (int n : iterations) {
        float accum = 0.0;
        auto start = chrono::high_resolution_clock::now();
        for (int i = 0; i < n; i++) {
            float res1, res2;
            thread t1([&]() { res1 = formula1(x); });
            thread t2([&]() { res2 = formula2(x); });
            t1.join();
            t2.join();
            float res3 = res1 + res2 - res1;
            accum += res3;
        }
        auto end = chrono::high_resolution_clock::now();
        chrono::duration<double> delta = end - start;
        cout << "Iterations (threads): " << n << endl;
        cout << "Time: " << delta.count() << " seconds" << endl;
        cerr << "Accumulated value: " << accum << endl;
    }
    return 0;
}

```

После компиляции и исполнения программы получаем следующие результаты:

```

Iterations (threads): 10000
Time: 1.33117 seconds
Accumulated value: 2600.09
Iterations (threads): 100000
Time: 15.8736 seconds
Accumulated value: 25980.4

```

3. (C++ PROCESS) Параллельные вычисления через процессы.

```
#include <iostream>
#include <chrono>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
using namespace std;
float formula1(float x) { return x*x-x*x+x*4-x*5+x+x; }
float formula2(float x) { return x+x; }
int main() {
    int iterations[] = {10000, 100000};
    float x = 0.13;
    float* shared_results = (float*)mmap(NULL, 2*sizeof(float),
                                          PROT_READ | PROT_WRITE,
                                          MAP_SHARED | MAP_ANONYMOUS,
                                          -1, 0);

    for (int n : iterations) {
        float accum = 0.0;
        auto start = chrono::high_resolution_clock::now();
        for (int i = 0; i < n; i++) {
            pid_t p1 = fork();
            if (p1 == 0) {
                shared_results[0] = formula1(x);
                return 0;
            }
            pid_t p2 = fork();
            if (p2 == 0) {
                shared_results[1] = formula2(x);
                return 0;
            }
            waitpid(p1, NULL, 0);
            waitpid(p2, NULL, 0);
            float res1 = shared_results[0];
            float res2 = shared_results[1];
            float res3 = res1 + res2 - res1;
            accum += res3;
        }
        auto end = chrono::high_resolution_clock::now();
        chrono::duration<double> delta = end - start;
        cout << "Iterations (process): " << n << endl;
        cout << "Time: " << delta.count() << " seconds" << endl;
        cerr << "Accumulated value: " << accum << endl;
    }
    munmap(shared_results, 2*sizeof(float));
    return 0;
}
```

Результат компиляции и исполнения программы:

```
Iterations (threads): 10000  
Time: 6.34458 seconds  
Accumulated value: 2600.09  
Iterations (threads): 100000  
Time: 66.6246 seconds  
Accumulated value: 25980.4
```